02244 Logic for Security
Security Protocols
Privacy

Sebastian Mödersheim

March 9, 2026

Challenges from Last Week

Channel Calculus Rule

- If we have a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$
- Then we can achieve a secure channel $B \bullet\!\!\rightarrow\!\!\bullet\, A$ in the opposite direction.

Channel Calculus Rule

- If we have an authenticated channel $A \bullet\!\!\rightarrow B$
- and a confidential channel $A \rightarrow\!\!\bullet\, B$.
- Then we can achieve a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$.

Challenges from Last Week

Channel Calculus Rule

- If we have a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$
- Then we can achieve a secure channel $B \bullet\!\!\rightarrow\!\!\bullet\, A$ in the opposite direction.

Possible solution:

$$\begin{array}{lll} A \bullet\!\!\rightarrow\!\!\bullet\, B: & K \\ B \rightarrow A: & \{\!|Payload|\!\}_K \end{array}$$

Channel Calculus Rule

- If we have an authenticated channel $A \bullet\!\!\rightarrow B$
- and a confidential channel $A \rightarrow\!\!\bullet\, B$.
- Then we can achieve a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$.

Challenges from Last Week

Channel Calculus Rule

- If we have a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$
- Then we can achieve a secure channel $B \bullet\!\!\rightarrow\!\!\bullet\, A$ in the opposite direction.

Possible solution:

$$\begin{array}{ll} A \bullet\!\!\rightarrow\!\!\bullet\, B: & K \\ B \rightarrow A: & \{\!|Payload|\!\}_K \end{array}$$

Channel Calculus Rule

- If we have an authenticated channel $A \bullet\!\!\rightarrow B$
- and a confidential channel $A \rightarrow\!\!\bullet\, B$.
- Then we can achieve a secure channel $A \bullet\!\!\rightarrow\!\!\bullet\, B$.

Possible solution:

$$\begin{array}{ll} A \rightarrow\!\!\bullet\, B: & Payload \\ A \bullet\!\!\rightarrow B: & h(Payload) \end{array}$$

Challenges from Last Week

- Challenge: Let s be an honest server. Suppose we have channels $s \bullet\!\!\rightarrow\!\!\bullet\, A$ and $s \bullet\!\!\rightarrow\!\!\bullet\, B$, can we achieve $A \bullet\!\!\rightarrow\!\!\bullet\, B$? Why does s have to be honest for that?

Challenges from Last Week

- Challenge: Let s be an honest server. Suppose we have channels $s \bullet\!\!\rightarrow\!\!\bullet\, A$ and $s \bullet\!\!\rightarrow\!\!\bullet\, B$, can we achieve $A \bullet\!\!\rightarrow\!\!\bullet\, B$? Why does s have to be honest for that?
- Possible Solution:

$$\begin{array}{ll} s \bullet\!\!\rightarrow\!\!\bullet\, A: & A, B, KAB \\ s \bullet\!\!\rightarrow\!\!\bullet\, B: & A, B, KAB \\ A \rightarrow B: & \{\!|Payload|\!\}_{KAB} \end{array}$$

Challenges from Last Week

- Challenge: Let s be an honest server. Suppose we have channels $s \bullet\!\!\rightarrow\!\!\bullet\, A$ and $s \bullet\!\!\rightarrow\!\!\bullet\, B$, can we achieve $A \bullet\!\!\rightarrow\!\!\bullet\, B$? Why does s have to be honest for that?
- Better solution:

$$\begin{array}{rl} A \bullet\!\!\rightarrow\!\!\bullet\, s: & PKA \\ B \bullet\!\!\rightarrow\!\!\bullet\, s: & PKB \\ s \bullet\!\!\rightarrow\!\!\bullet\, A: & A, B, PKA, PKB \\ s \bullet\!\!\rightarrow\!\!\bullet\, B: & A, B, PKA, PKB \\ A \rightarrow B: & \{\{B, Payload\}_{\mathrm{inv}(PKA)}\}_{PKB} \end{array}$$

  This has perfect forward secrecy: even if s later gets compromised, the intruder cannot read the Payload.

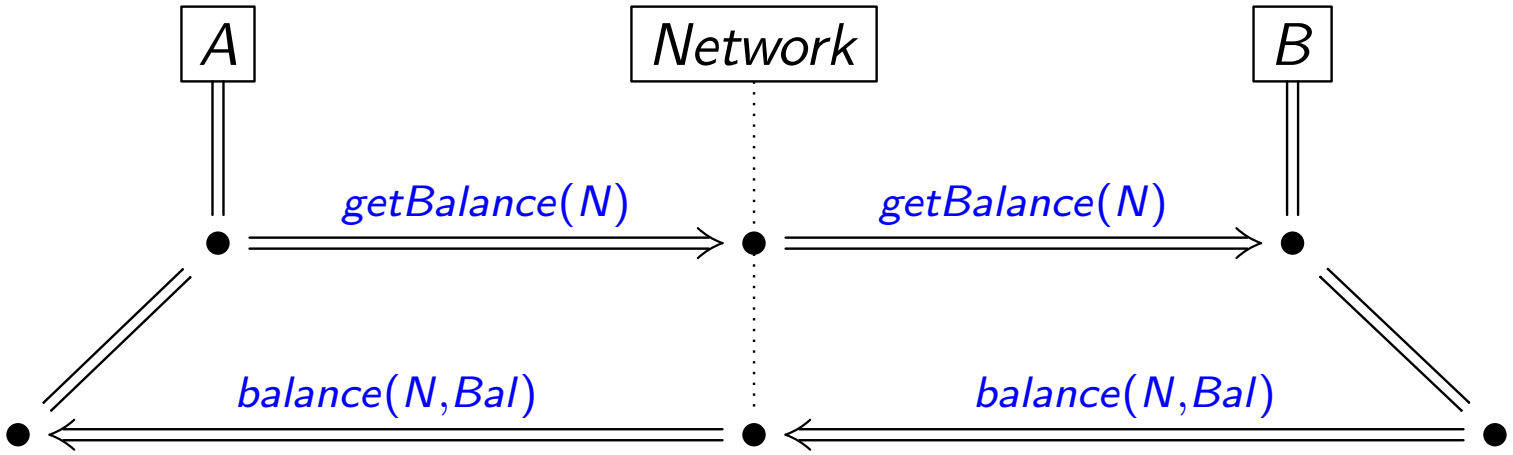
Challenges from Last Week

- Challenge: Let s be an honest server. Suppose we have channels $s \bullet\!\!\rightarrow\!\!\bullet\, A$ and $s \bullet\!\!\rightarrow\!\!\bullet\, B$, can we achieve $A \bullet\!\!\rightarrow\!\!\bullet\, B$? Why does s have to be honest for that?
- Better solution:

$$\begin{array}{rl} A \bullet\!\!\rightarrow\!\!\bullet\, s: & PKA \\ B \bullet\!\!\rightarrow\!\!\bullet\, s: & PKB \\ s \bullet\!\!\rightarrow\!\!\bullet\, A: & A, B, PKA, PKB \\ s \bullet\!\!\rightarrow\!\!\bullet\, B: & A, B, PKA, PKB \\ A \rightarrow B: & \{\{B, Payload\}_{\mathrm{inv}(PKA)}\}_{PKB} \end{array}$$

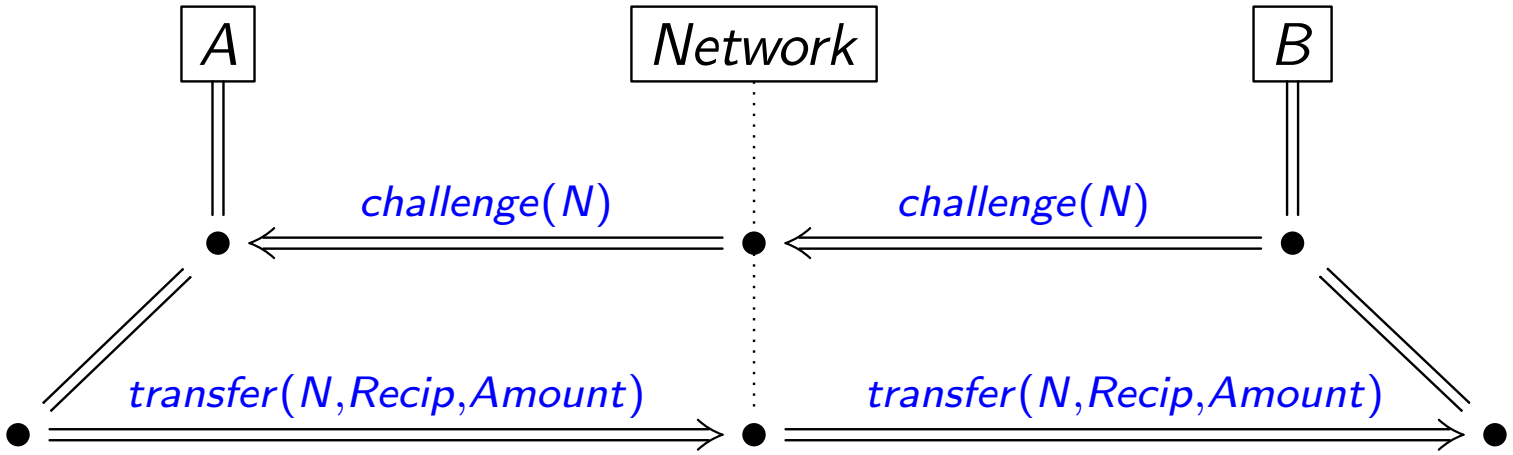
  This has perfect forward secrecy: even if s later gets compromised, the intruder cannot read the Payload.
- If we allow the intruder in place of the honest s we cannot achieve anything for a channel between A and B:
  - ★ All we can rely on are secure channels between A and i and between B and i.
  - ★ No information exchanged on these channels can build up anything reliable for the connection of A and B.

Challenges from Last Week



- Consider the banking example.
  - ★ Extend the banking application by transfer message where A can specified an amount of money and a recipient C.
  - ★ Protect the transfer against replay (without changing the channel).
  - ★ How could one implement a channel that has replay protection built in?

Challenges from Last Week



- Transfer as a message that names the Recipient and the Amount to transfer.
- Since the channel does not protect from replay, we can start with a challenge N from the bank to include in the transfer.
- Channel level: move this challenge response into the channel

$$B \to A: \quad N$$
$$A \to B: \quad \{\!|N, \mathit{Payload}|\!\}_{\mathsf{sk}(A,B)}$$

Challenges from Last Week

- Suppose A later claims she never made the transfer. Can B prove to an honest third party *judy* ("without reasonable doubt") that A indeed *did* make the transfer?

Challenges from Last Week

- Suppose A later claims she never made the transfer. Can B prove to an honest third party *judy* ("without reasonable doubt") that A indeed *did* make the transfer?
- In general, No: consider the transfer on the channel we have:
$$\{\!|\mathit{transfer}(N, \mathit{Recip}, \mathit{Bal})|\!\}_{\mathsf{sk}(A,B)}$$
  - ★ B can be sure this came from A.
  - ★ B cannot convince *judy* this came from A. He could have generated this himself.

Challenges from Last Week

- Suppose A later claims she never made the transfer. Can B prove to an honest third party *judy* ("without reasonable doubt") that A indeed *did* make the transfer?
- In general, No: consider the transfer on the channel we have:
$$\{\!|\mathit{transfer}(N, \mathit{Recip}, \mathit{Bal})|\!\}_{\mathsf{sk}(A,B)}$$
  - ★ B can be sure this came from A.
  - ★ B cannot convince *judy* this came from A. He could have generated this himself.
- This is why banks require a signature on the transfer!
  - ★ $\{\!|\{\mathit{transfer}(N, \mathit{Recip}, \mathit{Bal})\}_{\mathsf{inv}(\mathsf{pk}(A))}|\!\}_{\mathsf{sk}(A,B)}$
  - ★ What to do in case of online banking when customers don't have a digital signature?

Challenges from Last Week

- Suppose A later claims she never made the transfer. Can B prove to an honest third party *judy* ("without reasonable doubt") that A indeed *did* make the transfer?
- In general, No: consider the transfer on the channel we have:
$$\{\!|\mathit{transfer}(N, \mathit{Recip}, \mathit{Bal})|\!\}_{\mathsf{sk}(A,B)}$$
  - ★ B can be sure this came from A.
  - ★ B cannot convince *judy* this came from A. He could have generated this himself.
- This is why banks require a signature on the transfer!
  - ★ $\{\!|\{\mathit{transfer}(N, \mathit{Recip}, \mathit{Bal})\}_{\mathsf{inv}(\mathsf{pk}(A))}|\!\}_{\mathsf{sk}(A,B)}$
  - ★ What to do in case of online banking when customers don't have a digital signature?
  - ★ For instance: ask for confirmation via MitID.

Guessing Attacks

Example: a Variant of Microsoft-ChapV2

```
Protocol: PW
Types: Agent A,B;
       Number NB;
       Function pw,h;
Knowledge:
  A: A,B,pw(A,B),h;
  B: A,B,pw(A,B),h;
Actions:
  B->A: NB
  A->B: h(pw(A,B),NB)
Goals:
B authenticates A on NB
```

What if pw(A,B) has low entropy?

Low Entropy Passwords

- Low entropy passwords
  - ★ e.g. natural language words
  - ★ short

Low Entropy Passwords

- Low entropy passwords
  - ★ e.g. natural language words
  - ★ short
- The intruder compiles a dictionary D of passwords
  - ★ Relatively small space of passwords (a few million)
  - ★ Contains many weak passwords

Low Entropy Passwords

- Low entropy passwords
  - ★ e.g. natural language words
  - ★ short
- The intruder compiles a dictionary D of passwords
  - ★ Relatively small space of passwords (a few million)
  - ★ Contains many weak passwords
- Use D for a brute-force attack on an observed message:

Observed message	$nb, h(pw(a,b), nb)$
Construct	$nb, h(\ guess\ , nb)$ for every $guess \in D$

Low Entropy Passwords

- Low entropy passwords
  - ★ e.g. natural language words
  - ★ short
- The intruder compiles a dictionary D of passwords
  - ★ Relatively small space of passwords (a few million)
  - ★ Contains many weak passwords
- Use D for a brute-force attack on an observed message:

Observed message	$nb, h(pw(a,b), nb)$
Construct	$nb, h(\ guess\ , nb)$ for every $guess \in D$

- When $pw(a,b) = guess$ for some guess, the constructed and the observed message are identical, and the intruder has found out $pw(a,b)$.

Are weak passwords of any use?

```
Knowledge:
  A: A,B,pw(A,B),pk(B);
  B: A,B,pw(A,B),pk(B),inv(pk(B));
Actions:
  A->B: { A,pw(A,B),K }pk(B)
  B->A: {| NB |}K
Goals:
  B authenticates A on K
  A authenticates B on NB
  K secret between A,B
```

- This is similar to what happens in TLS-based login protocols.

Are weak passwords of any use?

```
Knowledge:
  A: A,B,pw(A,B),pk(B);
  B: A,B,pw(A,B),pk(B),inv(pk(B));
Actions:
  A->B: { A,pw(A,B),K }pk(B)
  B->A: {| NB |}K
Goals:
  B authenticates A on K
  A authenticates B on NB
  K secret between A,B
```

- This is similar to what happens in TLS-based login protocols.
- Guessing not possible – despite low-entropy $pw(A, B)$
  - ★ The intruder has to create $\{A, guess, K\}_{\mathrm{pk}(B)}$ for every guess.
    - ▸ That requires guessing K, too!

Are weak passwords of any use?

```
Knowledge:
  A: A,B,pw(A,B),pk(B);
  B: A,B,pw(A,B),pk(B),inv(pk(B));
Actions:
  A->B: { A,pw(A,B),K }pk(B)
  B->A: {| NB |}K
Goals:
  B authenticates A on K
  A authenticates B on NB
  K secret between A,B
```

- This is similar to what happens in TLS-based login protocols.
- Guessing not possible – despite low-entropy $pw(A, B)$
  - ★ The intruder has to create $\{A, guess, K\}_{\mathrm{pk}(B)}$ for every guess.
    - ▸ That requires guessing K, too!
- This is also why any reasonable implementation of asymmetric encryption contains randomization!
  - ★ $\{guessable\ message\}_{\mathrm{pk}(B)}$

Guessing Attacks in OFMC

```
B->A: NB
A->B: h(pw(A,B),NB)
Goals:
B authenticates A on NB
pw(A,B) guessable secret between A,B
```

Gives attack:

```
GOAL:
  guesswhat
...
ATTACK TRACE:
i -> (x20,1): x206
(x20,1) -> i: h(pw(x20,x25),x206)
i can produce secret h(guessPW,x206)

secret leaked: h(guessPW,x206)
```

Guessing Attacks in OFMC

Implementation in OFMC:

- Say $pw(A, B)$ is specified as a guessable secret between A and B.
- Let $guessPW$ be a constant known to the intruder.
- Check every message sent by an honest agent if it contains $pw(A, B)$.
  - ★ Example: outgoing message $h(pw(A, B), NA)$
  - ★ Then declare $h(guessPW, NA)$ as a secret between A and B.
  - ★ If the intruder is able to violate this secrecy goal, then he can perform a guessing attack.
- Additionally, if the password is used for symmetric encryption, then guessing the key is sufficient:
  - ★ Example: $\{\!|m|\!\}_{h(pw(A,B),NA)}$
  - ★ Then also $h(guessPW, NA)$ is a secret between A and B.

Extreme Views

An Example [Mark Ryan]

- Parents and their teenage daughter:
  - ★ The daughter wants to go out, but not tell the parents where she goes.
  - ★ The parents want to know where she is in case of an emergency.
  - ★ Both are legitimate interests!
  - ★ Danger: the parents may overreach – out of concern for their daughter – not respecting her privacy.

An Example [Mark Ryan]

- Parents and their teenage daughter:
  - ★ The daughter wants to go out, but not tell the parents where she goes.
  - ★ The parents want to know where she is in case of an emergency.
  - ★ Both are legitimate interests!
  - ★ Danger: the parents may overreach – out of concern for their daughter – not respecting her privacy.
- Non-electronic solution:
  - ★ The daughter writes where she is going onto letter and seals it.
  - ★ The parents can indeed open the letter case of an emergency.
  - ★ The parents are compelled not to open the seal unless there is an emergency.
- Technical solution: with a TPM

Privacy in Voting Systems

Example: FOO'92

FOO'92

- An online voting protocol by Fujioka, Okamoto, and Ohta.
- Relatively simple: uses standard crypto, few steps.
- Achieves some basic goals, but far from perfect.

We now develop this protocol in a few steps, and then use it as an example to describe privacy goals.

Development of FOO'92

Setup

- A population of voters $V_1, \ldots, V_N$.
- Each voter V_i has decided his or her vote $v_i \in \{0, 1\}$.
- r_i (and later b_i) are secret random numbers chosen by voter V_i.
- There is an administration A that controls who is a valid voter and issues the ballots.
- There is a counter C who collects all the ballots. C then publishes all ballots in a random order. For that, let π be permutation of the numbers $1, \ldots, N$.
- We assume anonymous channels similar to onion routing like TOR, and write $[A]\bullet\!\rightsquigarrow\!\bullet B$ for
  - ★ A has a secure channel with B, but with respect to a pseudonym of A, so B does not know A but can send a reply that only A receives.
  - ★ The intruder cannot observe that A and B have communicated.

Development of FOO'92

A Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\rightarrow \mathsf{all}$:	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Development of FOO'92

A Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Problem with this simplified version:

- The administrator A sees how everybody votes.
- Even though A would normally be an honest party, this is a bit much.

Development of FOO'92

A Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\rightarrow$ all :	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Problem with this simplified version:

- The administrator A sees how everybody votes.
- Even though A would normally be an honest party, this is a bit much.
- Idea: blind signatures.

Blind Signatures

Some asymmetric ciphers like RSA allow for the following:

- A user can produce a blinded value $blind(b, m)$ where m is the actual content, and b is a secret that only the user knows and without it, m cannot be read.
- One can have this signed by another party A: $\{blind(b, m)\}_{\mathsf{inv}(\mathsf{pk}(A))}$.
- The user can now unblind this signature by applying an unblind function with the secret b:

$$\begin{aligned} &unblind(b, \{blind(b, m)\}_{\mathsf{inv}(\mathsf{pk}(A))}) \\ &= \{unblind(b, blind(b, m))\}_{\mathsf{inv}(\mathsf{pk}(A))} = \{m\}_{\mathsf{inv}(\mathsf{pk}(A))}. \end{aligned}$$

- Thus, the user obtains a signature from A on a value m that A does not even know.

Development of FOO'92

A Less Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Development of FOO'92

A Less Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Problem with this less simplified version:

- If C is dishonest, it could suppress votes that it does not like.
- Could we further "blind" the votes, so that C must publish all ballots before knowning the vote they contain?

Development of FOO'92

A Less Simplified Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{blind(b_i, (v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{v_i, r_i\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{v[\pi[j]], r[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))}$

Problem with this less simplified version:

- If C is dishonest, it could suppress votes that it does not like.
- Could we further "blind" the votes, so that C must publish all ballots before knowning the vote they contain?
- Idea: commitment schemes.

Commitment Scheme

A commitment scheme is a bit similar to a symmetric encryption (with integrity):

- A user commits to a message m using a secret r, denoted $commit(m, r)$, and publishes this commitment.
- Later, the user reveals r:
  - ★ With r, everybody can open the $commit(m, r)$ and see m.
  - ★ Hiding: before revealing r, nobody can see m.
  - ★ Binding: there is no r' that opens the commitment to a different message m'. (At least it is not feasible to compute such r'.)

Development of FOO'92

The Real Version

Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet A$:	$\{blind(b_i, commit(v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(V_i))}$
	$A\bullet\!\rightsquigarrow\!\bullet[V_i]$:	$\{blind(b_i, commit(v_i, r_i))\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i.	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$\{commit(v_i, r_i)\}_{\mathsf{inv}(\mathsf{pk}(A))}$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{commit(v[\pi[j]], r[\pi[j]])\}_{\mathsf{inv}(\mathsf{pk}(A))}$
Each voter V_i finds their ballot in the published list,		
		i.e. j such that $\pi(j) = i$
	$[V_i]\bullet\!\rightsquigarrow\!\bullet C$:	$j, r[\pi[j]]$
For $j \in \{1, \ldots, N\}$.	$C \bullet\!\!\rightarrow$ all :	$\{commit(v[\pi[j]], r[\pi[j]])\}_{\mathsf{inv}(\mathsf{pk}(A))}, r[\pi[j]]$

Defining Privacy?

From the paper [FOO'92]:

- *Completeness* ... All valid votes are counted correctly.
- *Soundness* ..... The dishonest voter cannot disrupt the voting.
- *Privacy* ..... All votes must be secret.
- *Unreusability* .... No voter can vote twice.
- *Eligibility* ..... No one who isn't allowed to vote can vote.
- *Fairness* ..... Nothing must affect the voting.
- *Verifiability* ..... No one can falsify the result of the voting.

The Goals

How can we formulate the goals of such a protocol?

- Soundness: every vote that appears on the bulletin board is a valid vote, in particular it should forbid double voting and voter fraud.
  - ★ This is similar to an authentication goal with replay protection.
  - ★ It requires at least partial honesty of A (not issueing fake ballots).
- Completeness: no votes can be suppressed. A bit subtle...

The Goals

How can we formulate the goals of such a protocol?

- Soundness: every vote that appears on the bulletin board is a valid vote, in particular it should forbid double voting and voter fraud.
  - ★ This is similar to an authentication goal with replay protection.
  - ★ It requires at least partial honesty of A (not issueing fake ballots).
- Completeness: no votes can be suppressed. A bit subtle...
- Voter Privacy, sometimes called "secrecy of the vote".
  - ★ It is not a standard secrecy goal:
    - ▸ Any goal like v_i `secret between` V_i is trivially violated.

The Goals

How can we formulate the goals of such a protocol?

- Soundness: every vote that appears on the bulletin board is a valid vote, in particular it should forbid double voting and voter fraud.
  - ★ This is similar to an authentication goal with replay protection.
  - ★ It requires at least partial honesty of A (not issueing fake ballots).
- Completeness: no votes can be suppressed. A bit subtle...
- Voter Privacy, sometimes called "secrecy of the vote".
  - ★ It is not a standard secrecy goal:
    - ▶ Any goal like v_i `secret between` V_i is trivially violated.
    - ▶ Each v_i is either 0 or 1, and the intruder knows 0 and 1.

Defining Privacy Goals

Notions based on Indistinguishability:

- The intruder cannot tell whether V_4 has voted yes or no.

$$\boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 1} \sim \boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 0}$$

Defining Privacy Goals

Notions based on Indistinguishability:

- The intruder cannot tell whether V_4 has voted yes or no.

$$\boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 1} \sim \boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 0}$$

- The intruder cannot tell wether V_4 and V_5 have voted the same.

$$\boxed{v_4 = v_5} \sim \boxed{v_4 \neq v_5}$$

Defining Privacy Goals

Notions based on Indistinguishability:

- The intruder cannot tell whether V_4 has voted yes or no.

$$\boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 1} \sim \boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 0}$$

- The intruder cannot tell wether V_4 and V_5 have voted the same.

$$\boxed{v_4 = v_5} \sim \boxed{v_4 \neq v_5}$$

- Anything else?

Defining Privacy Goals

Notions based on Indistinguishability:

- The intruder cannot tell whether V_4 has voted yes or no.

$$\boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 1} \sim \boxed{\text{voter } V_4 \text{ voted } \quad v_4 = 0}$$

- The intruder cannot tell wether V_4 and V_5 have voted the same.

$$\boxed{v_4 = v_5} \sim \boxed{v_4 \neq v_5}$$

- Anything else?
- Unclear whether we have found all relevant distinguishability questions.
- Goals formulated on a technical level with cryptographic messages.

Idea

Inspiration/Idea

In zero-knowledge proofs we can usually specify a statement that is being proved.

- Definitely, that statement is revealed to the verifier
- The verifier (or others) should not learn anything else
- Everybody can draw conclusions from everything they learned

Can we do something logical in general for privacy?

α-β privacy

We specify two formulae (in Herbrand logic):

- α the high-level information we deliberately reveal to the intruder/verify/public
- β the technical information like cryptographic messages that are observable (including α)

Two alphabets:

- Σ the alphabet of β, includes "crypto/technical stuff"
- $\Sigma_0 \subseteq \Sigma$ the alphabet of α, only "non-technical stuff"

Intuition: from β I do not learn anything (except "technical" stuff) that is not implied by α already.

Model Theory

Definition

A model of a formula is an interpretation of all symbols that makes the formula true.

Formula $\alpha \equiv x[1], x[2], x[3] \in \{0, 1\} \wedge x[1] + x[2] + x[3] = 1$.
What are the models of α?

Model Theory

Definition

A model of a formula is an interpretation of all symbols that makes the formula true.

Formula $\alpha \equiv x[1], x[2], x[3] \in \{0, 1\} \wedge x[1] + x[2] + x[3] = 1$.
What are the models of α?

Model	$x[1]$	$x[2]$	$x[3]$
$\mathcal{I}_1$	0	0	1
$\mathcal{I}_2$	0	1	0
$\mathcal{I}_3$	1	0	0

Intuition: what worlds can we imagine in which α is true?

Model-Theoretic Formulation

Definition (Model-theoretic Definition of α-β-privacy)

Every Σ_0-model of α can be extended to a Σ-model of β.

Thus, β does not allow the intruder to rule out any model of α.

Example/FOO

$$\begin{aligned} \alpha &\equiv x[1], x[2], x[3] \in \{0,1\} \wedge x[1] + x[2] + x[3] = 1 \\ \beta &\equiv x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge \\ &\quad\ \pi \text{ is a permutation on } \{1,2,3\} \end{aligned}$$

Here α-β-privacy holds:

- Given a model of α, construct the model of π such that everything works out.

Model-Theoretic Formulation

Definition (Model-theoretic Definition of α-β-privacy)

Every Σ_0-model of α can be extended to a Σ-model of β.

Thus, β does not allow the intruder to rule out any model of α.

Example/FOO

$$\begin{aligned} \alpha &\equiv x[1], x[2], x[3] \in \{0,1\} \wedge x[1] + x[2] + x[3] = 1 \\ \beta &\equiv x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge \\ &\quad\ \pi \text{ is a permutation on } \{1,2,3\} \end{aligned}$$

Here α-β-privacy holds:

- Given a model of α, construct the model of π such that everything works out.

$\beta \equiv \ldots \wedge x_1 = x_2$ violates α-β-privacy.

Example: Vote Secrecy in FOO'92

- The information that is deliberately revealed to the intruder (and to the public) is:
  - ★ the number of cast votes, say 100,
  - ★ and the result of the vote, say 52 for 1, the rest 0.

Example: Vote Secrecy in FOO'92

- The information that is deliberately revealed to the intruder (and to the public) is:
  - ★ the number of cast votes, say 100,
  - ★ and the result of the vote, say 52 for 1, the rest 0.
- $\alpha \equiv v_1, \ldots, v_{100} \in \{0, 1\} \wedge v_1 + \ldots + v_{100} = 52.$

Example: Vote Secrecy in FOO'92

- The information that is deliberately revealed to the intruder (and to the public) is:
  - ★ the number of cast votes, say 100,
  - ★ and the result of the vote, say 52 for 1, the rest 0.
- $\alpha \equiv v_1, \ldots, v_{100} \in \{0, 1\} \wedge v_1 + \ldots + v_{100} = 52$.
- It is a violation of voter privacy if the intruder finds out anything about the votes that does not follow from α.
  - ★ E.g. $v_4 = 1$.
  - ★ E.g. $v_4 = v_5$.

Example: Vote Secrecy in FOO'92

- The information that is deliberately revealed to the intruder (and to the public) is:
  - ★ the number of cast votes, say 100,
  - ★ and the result of the vote, say 52 for 1, the rest 0.
- $\alpha \equiv v_1, \ldots, v_{100} \in \{0, 1\} \wedge v_1 + \ldots + v_{100} = 52$.
- It is a violation of voter privacy if the intruder finds out anything about the votes that does not follow from α.
  - ★ E.g. $v_4 = 1$.
  - ★ E.g. $v_4 = v_5$.
- The goal works also in the case of a unanimous vote, e.g. 100 cast votes, 100 for 1: then α implies $v_i = 1$ for every vote.

Advantages of α-β Privacy

Declarative Specification

- Modeler specifies only
  - ★ the protocol itself – this gives β, i.e., what cryptographic messages the intruder can see,
  - ★ and the information deliberately released – this is α.
- Both α and β are something one should think about anyway!
- When in doubt, be stingy on α
- and generous on β.
- Connection to static equivalence of frames: we can show how to encode α-β privacy into this more low-level formalism.

Voter Privacy in FOO'92

The voter privacy goal (for N cast votes and K votes for yes):

$$\alpha \equiv v_1, \ldots, v_N \in \{0, 1\} \wedge v_1 + \ldots + v_N = K$$

holds even if A and C are dishonest and collaborate.

Advantages of α-β Privacy

Stability under Background Knowledge

Scenario

- In a small village in rural Denmark, everybody votes conservative.
- One day, somebody from Copenhagen moves in, and in the next election, there is one vote for a left-wing party...

- Attack is actually independent of the voting system!
- The best voting system cannot prevent that you may have background knowledge about the world and combine that with your observations!
- Suppose the intruder has a background knowledge α_0.

Advantages of α-β Privacy

Stability under Background Knowledge

Scenario

- In a small village in rural Denmark, everybody votes conservative.
- One day, somebody from Copenhagen moves in, and in the next election, there is one vote for a left-wing party...

- Attack is actually independent of the voting system!
- The best voting system cannot prevent that you may have background knowledge about the world and combine that with your observations!
- Suppose the intruder has a background knowledge α_0.

Theorem

(α, β)-*privacy implies* $(\alpha \wedge \alpha_0, \beta \wedge \alpha_0)$ *privacy for every* $\alpha_0 \in \mathcal{L}_{\Sigma_0}$.

ICAO 9303 BAC

- Protocol between passport RFID-tag and card reader.
- `sk(x)`: key of tag `x` with card reader (optically from passp. data)
- Challenge `N` from passport to card reader, who sends it back encrypted as a response.
- Writing as an alpha-beta transaction
  - ★ we only look at examples, for full details see [Gondron et al., 2022]

```
Transaction Challenge:
* x in {t1,t2}.      # alpha: x is a tag
new N.               # nonce from the tag
send N.              # ... sent on a public channel
send session(x,N).   # a pseudo-message that
                     # stores the state of tag
send scrypt(sk(x),N). # the answer from the server
nil
```

ICAO 9303 BAC

The tag receiving the challenge from the server:

```
Transaction Response:
receive Session.      # should be session(X,N)
receive M.            # should be scrypt(sk(X),N)
try X = sfst(Session) in # get X
try N = ssnd(Session) in # get N
try NN = dscrypt(sk(X),M) # decrypt M
in State := noncestate[N]. # is the nonce fresh
   if N=NN and State = fresh # and the one expected?
   then noncestate[N]:=spent. # then mark as spent
        send ok. nil          # send ok message
   else send nonceErr  . nil # nonce check failed
catch   send formatErr . nil # decryption failed
catch              nil  # not valid session (ignore)
catch              nil  # not valid session (ignore)
```

Example Run

Suppose two `Challenge` transactions have been executed, say with tags `x1` and `x2`.
Thus $\alpha \equiv x1, x2 \in \{t1, t2\}$ and the intruder knowledge is:

```
I1 -> session(x1,N1)
I2 -> scrypt(sk(x1),N1)
I3 -> session(x2,N2)
I4 -> scrypt(sk(x2),N2)
```

Now executing the Transaction `Response` where the intruder chooses the following inputs:

```
Transaction Response:
receive Session.     # use message I1
receive M.           # use message I4
...
```

Example Run

What happens in this transaction then?

```
Transaction Response:
receive Session.      # session(x1,N1)
receive M.            # scrypt(sk(x2),N2)
try X = sfst(Session) in # X=x1
try N = ssnd(Session) in # N=N1
try NN = dscrypt(sk(X),M)
```

... what happens here depends on whether or not `x1=x2`!

if `x1=x2`:

```
State := ...
if N=NN and ... # false
then ... # won't happen
else send nonceErr. nil
```

if `x1/=x2`:

```
send formatErr . nil
```

Example Run

What happens in this transaction then?

```
Transaction Response:
receive Session.      # session(x1,N1)
receive M.            # scrypt(sk(x2),N2)
try X = sfst(Session) in # X=x1
try N = ssnd(Session) in # N=N1
try NN = dscrypt(sk(X),M)
```

... what happens here depends on whether or not x1=x2!

if x1=x2:

```
State := ...
if N=NN and ... # false
then ... # won't happen
else send nonceErr. nil
```

if x1/=x2:

```
send formatErr . nil
```

From the observed output message, the intruder knows if x1=x2!

Check out new tool

noname tool

Laouen Fernet, Sebastian Mödersheim, and Luca Viganò: *A Decision Procedure for Alpha-Beta Privacy for a Bounded Number of Transitions*, Computer Security Foundations 2024.

Link to noname tool and paper

Conclusions

- Privacy goals are more subtle than standard secrecy!
  - ★ Relatively complicated notions like (observational) equivalence.
  - ★ Both hard for the modeler and automated tools.
- α-β-privacy as an new way to specify more declaratively:
  - ★ what high-level information α we publish (or reveal to an intruder)
  - ★ and what low-level/cryptographic information β can be observed.
- Privacy as a reachability problem: can we reach a state where β allows for an interesting derivation that α does not imply?
- Relation between α-β and equivalence-based formulations.
- Very active research area!

Challenges

Example/FOO

$$\begin{array}{rcl} \alpha & \equiv & x[1], x[2], x[3] \in \{0,1\} \wedge x[1] + x[2] + x[3] = 1 \\ \beta & \equiv & x[\pi[1]] = 1 \wedge x[\pi[2]] = 0 \wedge x[\pi[3]] = 0 \wedge \\ & & \pi \text{ is a permutation on } \{1,2,3\} \end{array}$$

- Consider the model $x[1] = 0, x[2] = 1, x[3] = 0$ of α.
  - ★ Find a permutation $\pi : \{1,2,3\} \to \{1,2,3\}$ such that β holds.
- Consider the actual messages that C in FOO publishes,

$$\{commit(r[\pi[j]], v[\pi[j]]\}_{\mathsf{inv}(\mathsf{pk}(A))},\ r[\pi[j]]$$

  - ★ That is at position j in the publishing, we have the vote of some voter $\pi[j]$.
  - ★ Can a voter *prove* to the intruder which one is their vote?
  - ★ Is that a desirable property of the protocol?

Relevant Research Papers

- David Chaum. *Blind signatures for untraceable payments.* CRYPTO 1983.
- Atsushi Fujioka, Tatsuaki Okamoto, and Kazui Ohta. *A practical secret voting scheme for large scale elections.* AUSCRYPT 1992.
- Bruce Schneier and Mudge: *Cryptanalysis of Microsoft's PPTP Authentication Extensions (MS-CHAPv2)*, CQRE '99.
- Gavin Lowe: *Analysing Protocol Subject to Guessing Attacks*, Journal of Computer Security, 12(1), 2004.
- Paul Hankes Drielsma, Sebastian Mödersheim and Luca Viganò: *A Formalization of Off-Line Guessing*, LPAR 2004.
- Martín Abadi and Cédric Fournet: *Private authentication*, Theoretical Computer Science, 322(3), 2004
- Steve Kremer and Mark D. Ryan. *Analysis of an Electronic Voting Protocol in the Applied Pi-Calculus*. ESOP 2005.
- King Ables and Mark D. Ryan. *Escrowed data and the digital envelope*. TRUST 2010.
- Matthias Horst, Martin Grothe, Tibor Jager, Jörg Schwenk: *Breaking PPTP VPNs via RADIUS Encryption*, CANS 2016.

Relevant Research Papers

Alpha-Beta Privacy

- Sebastian Mödersheim and Luca Viganò. *Alpha-Beta Privacy*, In ACM Transactions on Privacy and Security, 2018.
- Sébastien Gondron and Sebastian Mödersheim. *Formalizing and Proving Privacy Properties of Voting Protocols.* ESORICS 2019.
- Sébastien Gondron, Sebastian Mödersheim and Luca Viganò: *Privacy as Reachability*. CSF 2022, Open Access.
- Laouen Fernet, Sebastian Mödersheim, and Luca Viganò: *A decision procedure and typing result for alpha-beta privacy*, JCS 2025, 34(1).
  Link to `noname` tool and paper